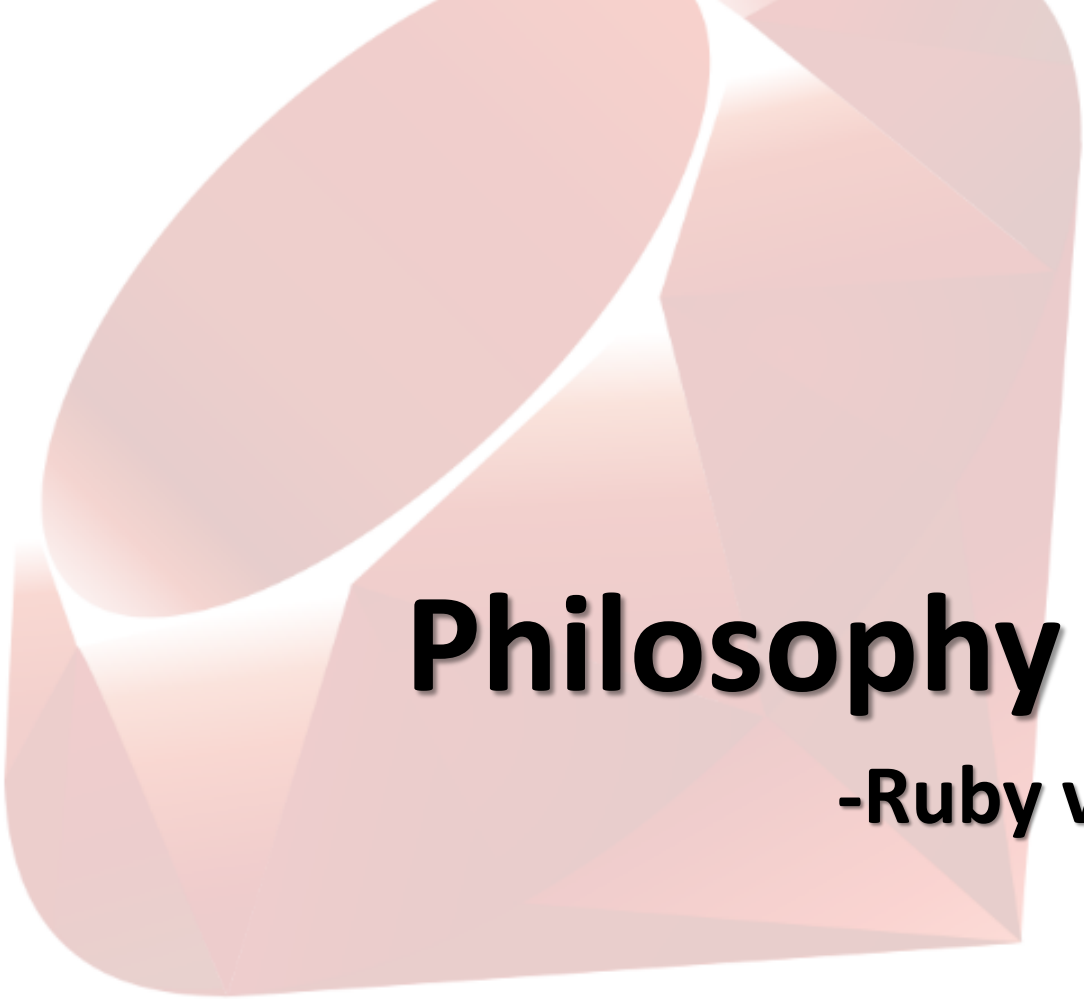




Philosophy of Static Typing

-Ruby vs. TypeScript-

Composed By: Hayato Ishida

For Japanese version, click [here](#)

Updated On: 24 June 2025

About Me

- Accounts
 - GitHub: [@hayat01sh1da](#)
 - X: [@hayat01sh1da](#)
 - LinkedIn: [@hayat01sh1da](#)
 - Speaker Deck: [@hayat01sh1da](#)
 - Docswell: [@hayat01sh1da](#)
- Occupation: Software Engineer
- Things I Am Into
 - Linguistics
 - Singing at Karaoke
 - Listening to Music
 - Watching Movies
 - Cat Feeding



Licences / Certifications

- English
 - TOEIC® Listening & Reading 915: Certified on December 2019
- Engineering
 - Information Security Management: Certified on November 2017
 - Applied Information Technology Engineer: Certified on June 2017
 - Fundamental Information Technology Engineer: Certified on November 2016
 - IT Passport: Certified on April 2016
- Others
 - Abacus 2nd Class: Certified on June 2002
 - Mental Arithmetic 3rd Class: Certified on February 2001

Skill Stack

- Languages
 - Japanese: Native Proficiency
 - English: Full professional Proficiency
- Development
 - **Backend: Ruby(Ruby on Rails), Python**
 - Frontend: TypeScript + React.js, TypeScript +Vue.js
 - **Database: MySQL**, PostgreSQL, MongoDB
 - **Architecture: Monolith, Modular Monolith**
 - Hosting: AWS ESK
 - Versioning: Git, GitHub
 - **CI/CD: GitHub Actions**, ArgoCD
 - Monitoring: Datadog, Sentry

VS.

TS

Work Histories

Period	Genre	Position	Jobs
Aug 2021 - Present	SaaS(Educational Service)	Software Engineer	• Backend Development • Frontend Development • CI/CD Maintenance • Weekly Release Manager • Troubleshooting • Documentation • Technical Debt Resolution • Tech Blog Entries • LT Speaker
Feb 2020 - Dec 2020	SaaS(Chatbot Platform)	Backend Engineer	• Backend Development • Documentation • Technical Debt Resolution • Inspection of Alternative API

Work Histories

Period	Genre	Position	Jobs
Jun 2018 - Jan 2020	Contract-basis Developer	Software Engineer	• Backend Development • Frontend Development • Troubleshooting • Quality Assurance • Tech Blog Entries
Apr 2016 - Jan 2018	System Engineering Service	System Engineer	• Corporate Account Admin • Windows Server Maintenance • Security Admin Assistant • Internationalisation / Localisation

International-Exchange Activities

- Activities in University
 - English Linguistics Seminar(Focusing on Mass Media English)
 - International-Exchange Clubs(The 2nd Year)
 - [International-Exchange Programmes conducted by Japan Cabinet Office](#)(2013 - 2016)
 - Japanese Linguistics Course(The Final Year)
- Overseas Life Experience
 - Working Holiday in Australia(April 2014 - March 2015)
 - Language School for 1 month in Sydney
 - Work for 6 Months in [Hamilton Island Resort](#)
 - Volunteering for 1 Month as Assistant Teacher of Japanese Language at [St Ives High School](#) in NSW
- Other Activities
 - Keep Everyday Journal in English (April 2014 - Present)
 - [Sunrise Toastmasters Club](#)(February 2017 - March 2018)
 - [Vital Japan](#)(January 2018 - July 2019, October 2022 - February 2023)
 - Self Learning and Training of English Language
 - Participation Ruby-Related Tech Conferences

VS.

T S

Agenda

1. [The Basics of Static Typing](#)
2. [The Typing Ecosystem](#)
3. [The Typing Philosophy](#)
4. [Summary](#)
5. [References](#)

VS.

The TypeScript logo, consisting of a light blue rounded square containing the letters 'TS' in a large, white, sans-serif font.

TS



1. The Basics of Static Typing

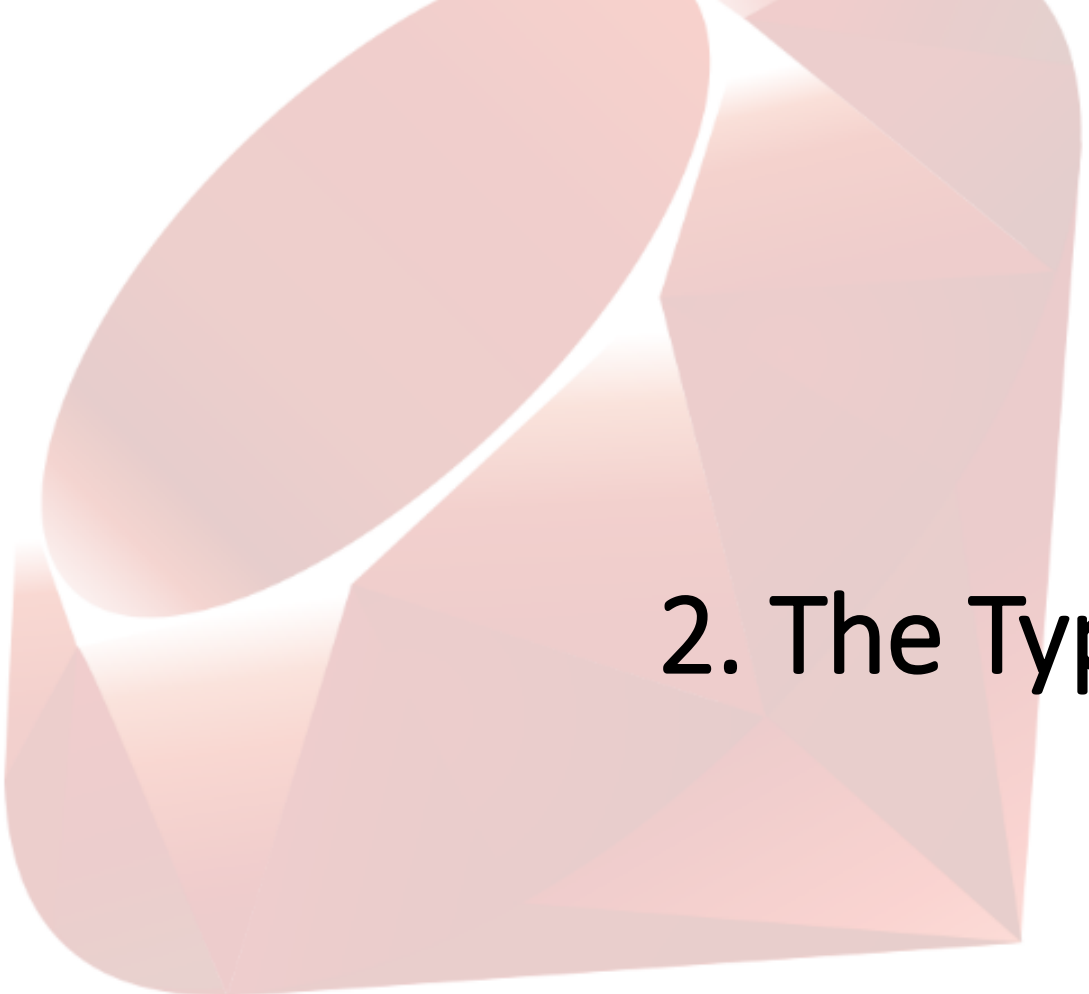
1. The Basics of Static Typing

Trade-offs

- 👍 Protects a system and/or program from **ABEND(Abnormal End)** and **security holes** caused by **Undefined Behaviours**
- 👍 Detects **TypeError** prior to a code execution
- 👍 Guards **large-scale codebase more efficiently** from such *Nasal Demons* as **bugs, type inconsistency and unsafe refactoring**
- 👎 **Worsens performance due to a huge overhead** because it checks the type of a receiver every time in some processing system(e.g., JavaScript), such as ``x.foo()``
- 👎 Forces developers to **maintain not just codes but type declarations**
- 👎 Brings developers **another learning costs to master how to make it work**

1. The Basics of Static Typing

Category	Timing of Type Inspection	Language
Static Typing	Prior to code execution	• TypeScript • C • Rust • <u>Java</u> • <u>Swift</u> etc.
Dynamic Typing	During the code execution	• Ruby • JavaScript • <u>Java</u> • <u>Swift</u> etc.



2. The Typing Ecosystem



TS

2. The Typing Ecosystem

Ruby

System	Gem	Description
RBS	RBS (Ruby Signature)	An official gem of which language to describe the structure of Ruby programs. Not DSL of Ruby but another language.
	Steep	Checks types according to definitions on RBS
	TypeProf	Infers definitions in RBS according to Ruby codes
	rbs-inline	Generates RBS by inline comments on Ruby codes
RBI	RBI (Ruby Interface)	Provides a Ruby API to compose Ruby I/F files for Sorbet
	Sorbet	Provides existing Ruby codes with gradual typing and quick response with errors and suggestions
	Tapioca	Compiles RBI files via not just Ruby source codes but Gems, Rails and other DSLs

2. The Typing Ecosystem

Ruby: RBS + Steep

e.g. RBS + Steep

person.rb

```
1 require_relative './phone'
2 require_relative './email'
3
4 class Person
5   attr_reader :name, :contacts
6
7   def initialize(name:)
8     @name = name
9     @contacts = []
10  end
11
12 def guess_country()
13   contacts.map {
14     |it| it.country if it.is_a?(Phone)
15   }.compact.first
16 end
17 end
```

VS.

sig/person.rbs

```
1 class Person
2   @name: untyped
3
4   @contacts: untyped
5
6   attr_reader name: untyped
7
8   attr_reader contacts: untyped
9
10  def initialize: (name: untyped) -> void
11
12  def guess_country: () -> untyped
13 end
```

\$ rbs prototype rb person.rb

\$ steep check
Type checking files:
...
No type error detected. 🍵

2. The Typing Ecosystem

Ruby: RBS + Steep
e.g. rbs-inline + Steep

person.rb

```
1 #rbs_inline::enabled
2 require_relative './phone'
3 require_relative './email'
4
5 class Person
6   attr_reader :name #: String
7   attr_reader :contacts #: Array[Phone | Email]
8
9   # @rbs::name: String
10  # @rbs::addresses: Array[String]
11  # @rbs::return: void
12  def initialize(name:)
13    @name = name
14    @contacts = []
15  end
16
17  # : () -> (String | nil)
18  def guess_country()
19    contacts.map {
20      |it| it.country if it.is_a?(Phone)
21    }.compact.first
22  end
23 end
```

\$ rbs-inline --output person.rb
🎨 Generated 1 RBS files under
sig/generated

\$ steep check
Type checking files:
...
No type error detected. 🍵

sig/generated/person.rbs

```
1 class Person
2   @name: String
3
4   @contacts: Array[Phone | Email]
5
6   attr_reader name: String
7
8   attr_reader contacts: Array[Phone | Email]
9
10  def initialize:(name: String) -> void
11
12  def guess_country: () -> (String | nil)
13 end
```

2. The Typing Ecosystem

TypeScript

On one hand, Ruby has not embraced another ecosystem than RBS yet in order to keep backward compatibility in terms of its syntax.

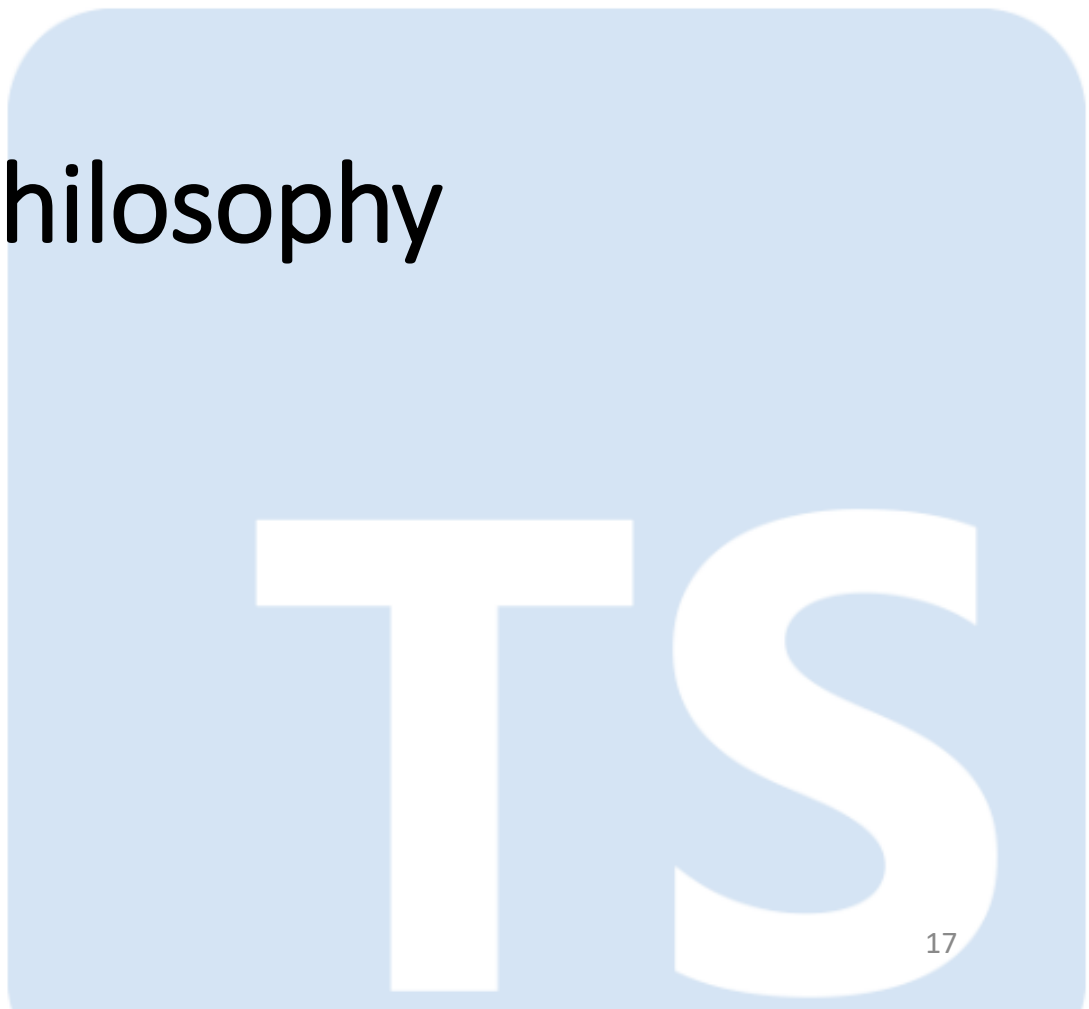
On the other hand, TypeScript dynamically changed JavaScript's ecosystem by introducing [Generics](#), [Object Types](#) and [Interfaces](#).

The drastic update broke an implicit assumption that we must introduce type check **without making any differences in existing code**, mentioned Mr. Sotaro Matsumoto([@sotaro](#)), a Ruby Committer, in [a blog article](#).

He also referred to the fact that it was because JavaScript had already been well-organised in terms of **its rich libraries and development environment with high functionality, which made it possible to change existing JavaScript codebase to that of TypeScript**.



3. The Typing Philosophy



TS

3. The Typing Philosophy

Ruby

In [a blog article](#) posted by a Developer Advocate at Heroku [Mr. Jonan Scheffler](#), the creator of Ruby Mr. Yukihiro Matsumoto, generally called Matz, mentioned the introduction of static typing to Ruby as follows.

It proves his philosophy of Ruby which **thinks highly of rich coding experience leading to developer's happiness and backward compatibilities.**

Jonan) *So, along with the JIT, you've also talked about some type changes to coming to Ruby 3 and the optional static types. Can you tell us about that?*

Matz) *Yeah, the third major goal of the Ruby 3 is adding some kind of static typing while **keeping the duck typing**, so some kind of structure for soft-typing or something like that. The main goal of the type system is to **detect errors early**. So adding this kind of **static type check or type interfaces does not affect runtime.***

3. The Typing Philosophy

Ruby

In fact, [RBS](#) is independent ecosystem of Ruby itself and the signature is a different language from it.

`xxx.rbs` is separate from the corresponding `xxx.rb` and [rbs-inline](#) treats type definitions as comment so does not affect existing Ruby codes while it provides more meanings than semantic description.

In my opinion, the concept of [rbs-inline](#) is very developer-friendly as follows.

- Static typing plays a role in a document
- The type of return value should be articulated especially on an app used as a web API
- Inline comments of [rbs-inline](#) is literally "get two for the price of one"

3. The Typing Philosophy

TypeScript

On p.3 of his book *型システムのしくみ - TypeScript で実装しながら学ぶ型とプログラミング言語*, Mr. Yusuke Endo, a Ruby committer, mentions the philosophy that TypeScript focuses only on detection of undesirable behaviours as long as **it is reasonable and does not drastically change the traditional JavaScript coding experiences**.

For example, Both of afterword variable definition below raise no error in type checking, but the latter raises **ReferenceError** in execution.

a.ts

```
1 const f = () => x;
2 const x = 1;
3 console.log(f());
```

b.ts

```
1 const f = () => x;
2 console.log(f());
3 const x = 1;
```

	WSL
Check file:// a.ts	\$ deno check a.ts
1	\$ deno run -A a.ts
Check file:// b.ts	\$ deno check b.ts
	\$ deno run -A b.ts
error: Uncaught (in promise) ReferenceError: Cannot access 'x' before initialization	
const f = () => x;	
at f (file:// b.ts:1:17)	
at file:// b.ts:2:13	

3. The Typing Philosophy

TypeScript

This phenomenon illustrates **uninitialized variables can be referred to by already defined functions.**

Such a type unsafety should be detected in type-check system, but TypeScript is willing to ignore it.

Why?

According to the author, JavaScript has traditionally allowed this kind of implementation, so **restriction on uninitialized variables in TypeScript can ruin of backward compatibilities with JavaScript.**

It proves **TypeScript values traditional coding experiences** as well as Ruby.

4. Summary

vs.



TS

4. Summary

- Both Ruby and TypeScript try hard to guarantee traditional coding experiences by keeping backward compatibilities
- TypeScript succeeded in integration of static typing ecosystem by dynamic introduction of [Generics](#), [Object Types](#) and [Interfaces](#)
- Ruby has 2 big streams of static typing system, [RBS](#)(Ruby Signature) and [RBI](#)(Ruby Interface) ecosystems. No winner of Natural Selection and Survival of the Fittest yet
- In any case, developers can enjoy a lot more benefits by static typing than they expect, so it is highly worth of installation



5. References

vs.



TS

5. References

- [いかにして動的型付けの Ruby に静的な型検査を持ち込むか？ Steep と RBS が目指すもの](#)
 - Last Accessed On: 06 June 2025
- [RubyKaigi 2025 初参加エンジニアのための Ruby 静的型付けガイド](#)
 - Last Accessed On: 06 June 2025
- [The State of Static Typing in Ruby in 2025](#)
 - Last Accessed On: 06 June 2025
- [Why Static Typing is Essential for Efficient Ruby and Rails Development](#)
 - Last Accessed On: 06 June 2025
- [Static Typing in Ruby: It's Not a Cult, It's a Choice \(You Should Probably Make\)](#)
 - Last Accessed On: 06 June 2025

5. References

- [Ruby 3 × 3: Matz, Koichi, and Tenderlove on the future of Ruby Performance](#)
 - Last Accessed On: 06 June 2025
- [RBS](#)(Ruby Signature)
 - Last Accessed On: 06 June 2025
- [Steep](#)
 - Last Accessed On: 06 June 2025
- [TypeProf](#)
 - Last Accessed On: 06 June 2025
- [rbs-inline](#)
 - Last Accessed On: 06 June 2025
- [RBI](#)(Ruby Interface)
 - Last Accessed On: 06 June 2025

VS.

T S

5. References

- [Sorbet](#)
 - Last Accessed On: 06 June 2025
- [Tapioca](#)
 - Last Accessed On: 06 June 2025
- [TypeScript Official Documents](#)
 - Last Accessed On: 06 June 2025
- [10 Must-Know TypeScript Features to Improve Your Coding Efficiency](#)
 - Last Accessed On: 06 June 2025
- Yusuke Endo, *型システムのしくみ - TypeScript で実装しながら学ぶ型とプログラミング言語*, Tokyo, Lambda Note Ltd., 2025

VS.



EOD

